OFFICIAL PRODUCT DOCUMENTATION

Yana OS

Complete platform for electric vehicle rider operations — onboarding, fleet, payments, marketplace, skills & support.

VERSION	DOCUMENT TYPE	LAST UPDATED
1.0.0 (Stable)	User Manual & Technical Ref	April 2026
AUDIENCE	**PLATFORM**	**STATUS**
Admins, Ops, Developers	Web + Android/iOS	Production Ready

Auth Service · Rider Service · Fleet Service · Payments Service · Marketplace Service · Skills Service · Support Service · Maintenance Service · Admin Dashboard · Rider Mobile App

NAVIGATION

Table of Contents

Click any section to navigate directly to it.

1 Executive Summary

2 System Architecture

3 Admin Dashboard Manual

4 Rider Mobile App Manual

5 Data Architecture & Schemas

6 Data Relationships

7 Business Workflows

8 Technology Stack

9 Infrastructure & Deployment

10 API Gateway & Routing

SECTION 1

Executive Summary

What Yana OS is, who uses it, and the problems it solves.

Yana OS is a production-grade, microservices-based operations platform for managing a fleet of electric vehicles and the riders who operate them. It provides a complete digital backbone for EV-based gig workforce companies — from recruiting and verifying riders, to assigning vehicles, scheduling demand work shifts, collecting daily rent, training riders, and resolving support issues.

Rider Lifecycle

End-to-end digital onboarding with KYC verification, document management, and status state machine from application to activation.

Fleet Management

Track every vehicle — GPS telemetry, battery health, allotments, maintenance schedules, and hub utilization.

Payments & Wallet

Immutable double-entry ledger, automated daily rent collection, Razorpay/UPI autopay, incentives, and penalties.

Demand Marketplace

Client demand slot publishing, smart rider matching, attendance check-in/out with GPS, and automated earnings calculation.

Skills & Training

Multilingual video training modules with quizzes, badges, points, streaks, and global leaderboard gamification.

Support

Full ticket lifecycle with SLA tracking, multi-channel messaging, WhatsApp integration, and satisfaction ratings.

Primary User Roles

ROLE	PLATFORM	PRIMARY RESPONSIBILITIES
`SUPER_ADMIN`	Admin Dashboard	Full system access — all modules, user management, financial controls
`CITY_ADMIN`	Admin Dashboard	City-scoped operations — riders, fleet, marketplace for their city
`HUB_OPS`	Admin Dashboard	Hub-level fleet operations — vehicle allotments, maintenance, returns
`SALES`	Admin Dashboard	Client management, demand slot creation, marketplace analytics
`SUPPORT_AGENT`	Admin Dashboard	Ticket resolution, rider communication, KYC document review
`VIEWER`	Admin Dashboard	Read-only access to dashboards and reports
`Rider`	Mobile App (Flutter)	View jobs, apply for slots, check wallet, watch training, raise tickets

Platform Components

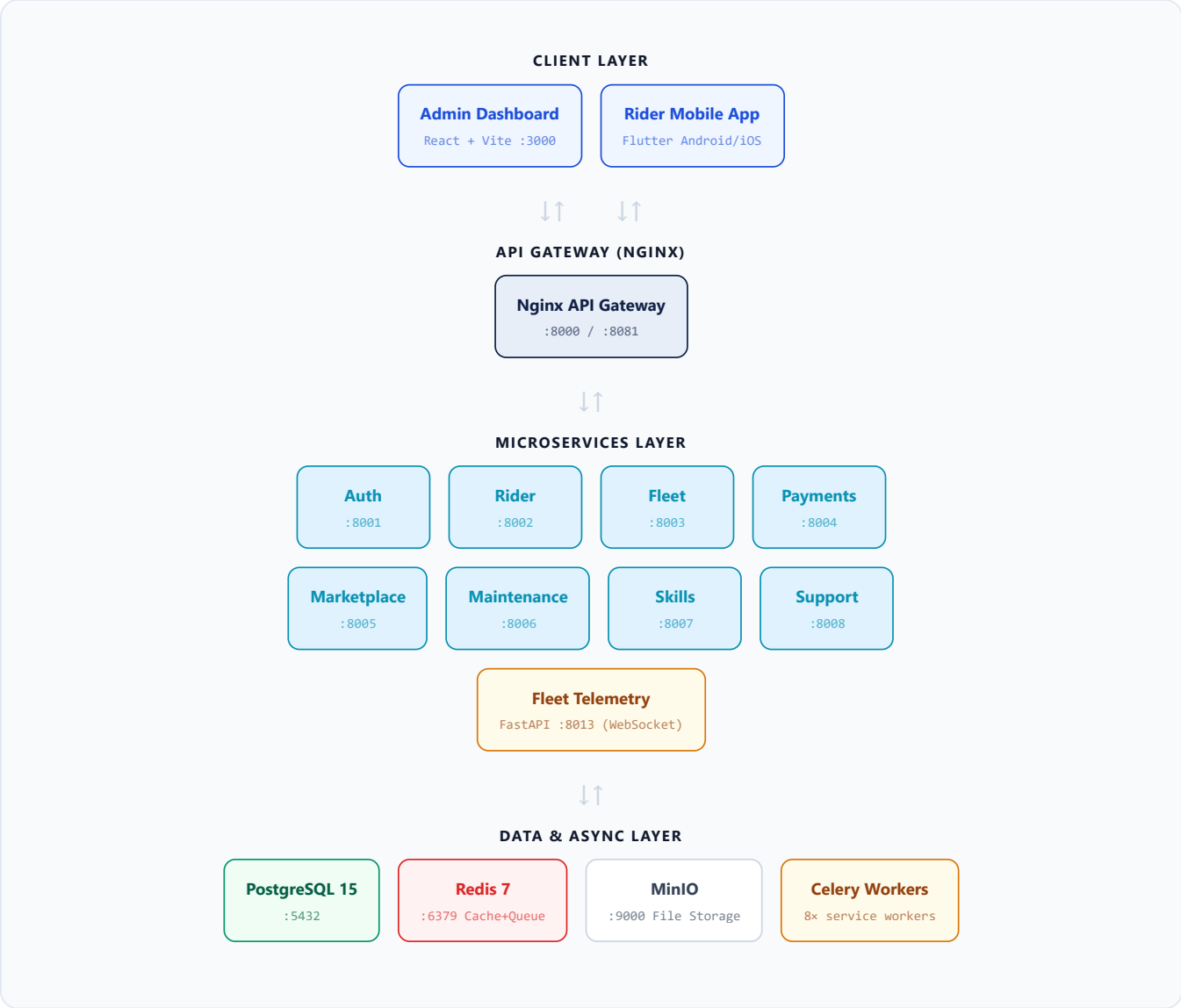
COMPONENT	TECHNOLOGY	URL / ACCESS
`Admin Dashboard`	React 18 + Vite + Tailwind	`http://localhost:3000`
`Rider Mobile App`	Flutter 3.41 + Riverpod	Android / iOS APK
`API Gateway`	Nginx	`http://localhost:8000`
`API Documentation`	Swagger / OpenAPI	`http://localhost:800X/api/docs/`
`MinIO Console`	S3-Compatible Storage	`http://localhost:9001`
`Database`	PostgreSQL 15	Port 5432 (internal)
`Cache / Queue`	Redis 7	Port 6379 (internal)

SECTION 2

System Architecture

How all components connect, route, and communicate with each other.

Yana OS follows a **microservices architecture** where each domain is an independent Django service. All traffic is routed through an Nginx API Gateway. Asynchronous tasks are handled by Celery workers backed by Redis. File storage uses MinIO (S3-compatible). All services share a single PostgreSQL 15 database with separate schemas.



Request Flow

Every API call follows this path:

1. **Client Request** — Browser / App
2. → **Nginx Gateway** — Rate limiting, routing
3. → **Auth Middleware** — JWT validation
4. → **Django Service** — Business logic
5. → **PostgreSQL** — Data persistence
6. → **JSON Response** — success + data

Standard API Response Format

All services use a consistent response envelope:

```json
{
  "success": true,
  "data": {
    // Response payload — object or list
  }
}
```

```
// Error response:
{
  "success": false,
  "error": {
    "message": "Human-readable error",
    "code":    "MACHINE_READABLE_CODE"
  }
}
```

SECTION 3

Admin Dashboard — User Manual

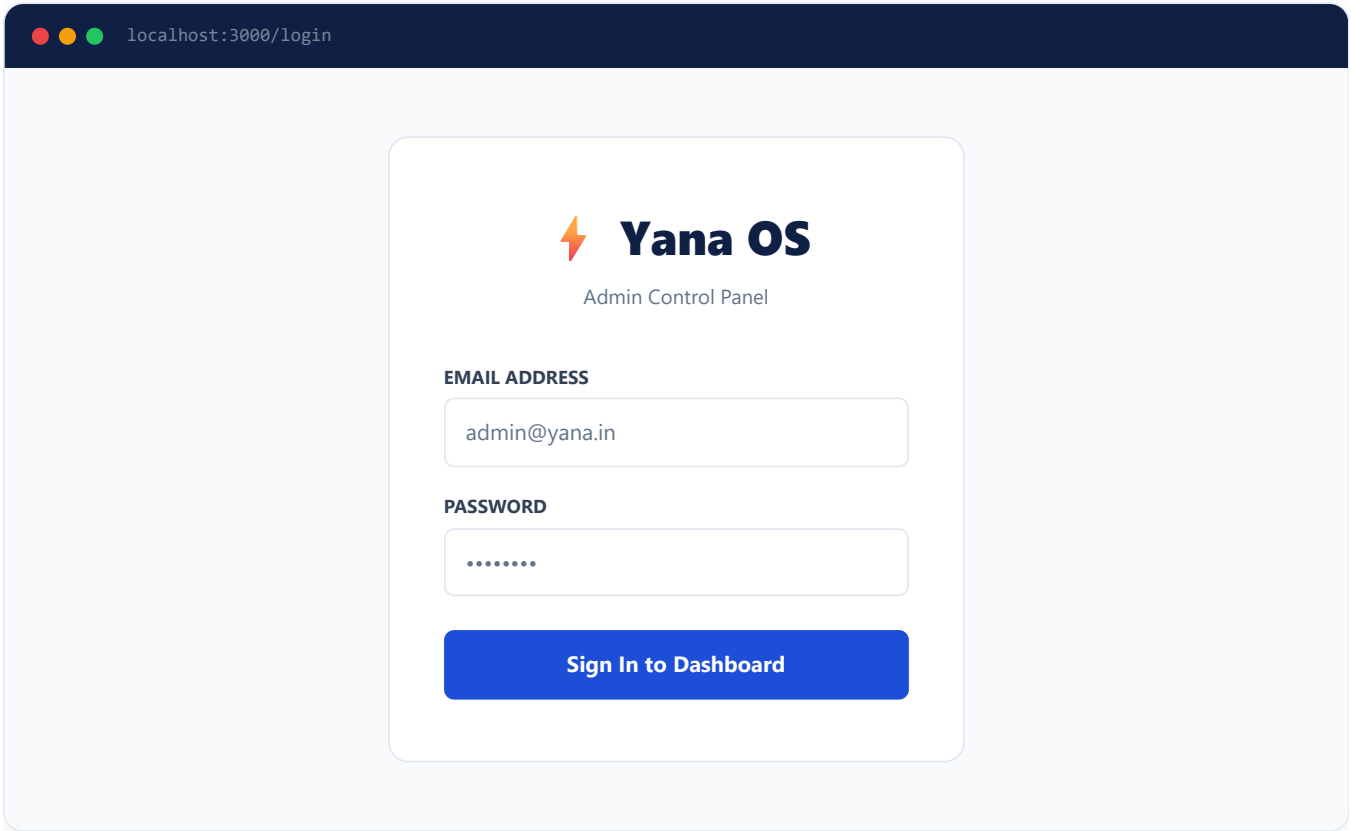
Complete guide to every page, tile, filter, dropdown and action in the Admin Dashboard.

ACCESS

The Admin Dashboard runs at `http://localhost:3000`. It is built with React 18, Vite, and Tailwind CSS. All data is fetched live from the backend API Gateway at `http://localhost:8000`.

3.1 — Login Page

The login page is the entry point for all admin users. It authenticates against the Auth Service using email and password.



Fields & Actions

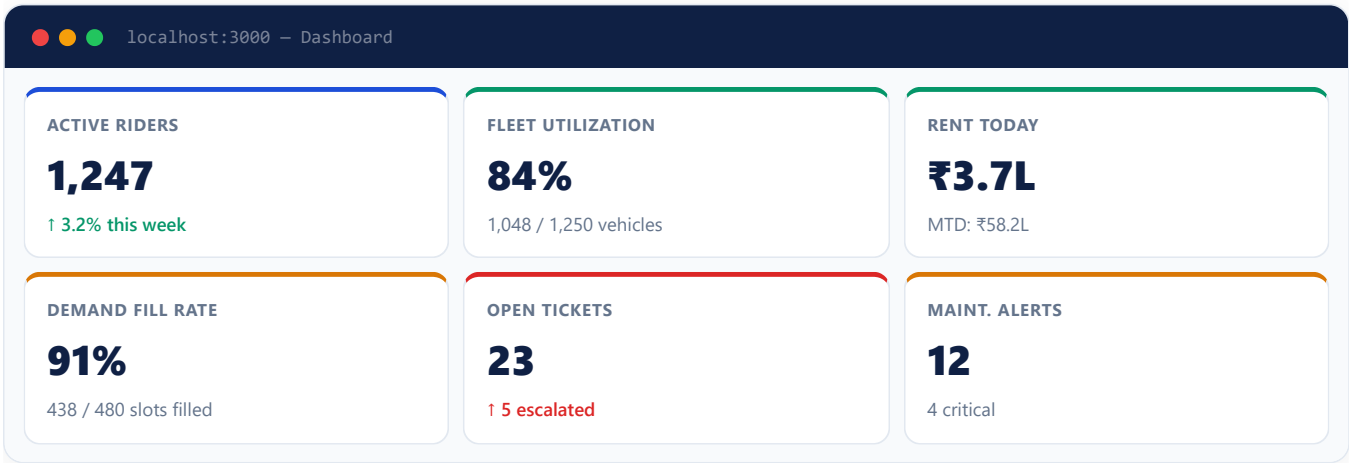
FIELD / CONTROL	TYPE	DESCRIPTION
Email Address	Text input	Admin account email. Seeded accounts: admin@yana.in, ops@yana.in, sales@yana.in
Password	Password input	Admin account password. Default seeded passwords: Admin@123, Ops@123, Sales@123
Sign In button	Action	Calls `POST /api/v1/auth/admin/login`. On success, stores JWT access and refresh tokens in localStorage. Redirects to Dashboard.

SESSION MANAGEMENT

Access tokens expire in 1 hour. The dashboard automatically refreshes them using the stored refresh token. If both expire, the user is redirected to login.

3.2 — Dashboard Overview (Control Tower)

The main dashboard provides a real-time operational snapshot of the entire Yana OS platform. It is the first screen after login and serves as a command center for operations teams.



Dashboard Tiles — Purpose & Rationale

TILE	SOURCE API	METRIC CALCULATED AS	BUSINESS PURPOSE
`Active Riders`	Rider Service	COUNT(riders WHERE status='ACTIVE')	Operational capacity indicator. Falling numbers signal churn or KYC backlog.
`Fleet Utilization`	Fleet Service	ALLOCATED / (AVAILABLE + ALLOCATED) × 100	Asset ROI indicator. Below 75% signals under-allocation; above 95% signals shortage risk.
`Rent Today`	Payments Service	SUM(wallet_ledger WHERE payment_type=DAILY_RENT AND accounting_date=today)	Daily revenue health check. Compare to expected (active riders × daily rent).
`Demand Fill Rate`	Marketplace Service	SUM(riders_confirmed) / SUM(riders_required) × 100 for today's slots	Client SLA adherence. Below 85% triggers manual outreach to riders.
`Open Tickets`	Support Service	COUNT(tickets WHERE status IN ('OPEN','ASSIGNED','IN_PROGRESS'))	Support queue health. Escalated count shown separately for manager awareness.
`Maintenance Alerts`	Fleet/Maintenance Service	COUNT(alerts WHERE is_acknowledged=False)	Fleet safety indicator. Critical alerts (battery, brakes) require same-day action.

Dashboard Charts

CHART	TYPE	DATA	USAGE
`Revenue Trend`	Line chart (30 days)	Daily rent + incentives credited	Track revenue growth; identify collection dips
`Fleet Status`	Donut chart	Vehicle counts by status	Visualize available vs allocated vs maintenance vehicles
`Hub Utilization`	Horizontal bar	Per-hub utilization %	Identify underperforming hubs needing attention
`Demand vs Attendance`	Grouped bar	Required vs confirmed vs shown-up riders per day	Identify no-show patterns and slot fill efficiency

3.3 — Riders Module

The Riders module is the central hub for managing the rider workforce. It covers the complete lifecycle from application through activation, KYC verification, document management, and offboarding.

3.3.1 Riders List View

UI CONTROL	TYPE	VALUES / OPTIONS	EFFECT
Search Bar	Text input	Name, phone, email	API param: `?q=search_term` — Filters riders by name/phone/email
Status Filter	Dropdown (multi-select)	APPLIED, DOCS_SUBMITTED, KYC_PENDING, KYC_FAILED, VERIFIED, TRAINING, ACTIVE, SUSPENDED, OFFBOARDED	API param: `?status=ACTIVE,TRAINING`
City Filter	Dropdown	List of active cities	API param: `?city_id=UUID`
Hub Filter	Dropdown (dependent on city)	Hubs in selected city	API param: `?hub_id=UUID`
KYC Status Filter	Dropdown	PENDING, SUBMITTED, UNDER_REVIEW, VERIFIED, REJECTED	API param: `?kyc_status=UNDER_REVIEW`
Date Range	Date pickers	Start/end date	Filter by rider created_at date range
Page Size	Dropdown	10, 25, 50, 100	API param: `?page_size=25`
Export CSV	Button	—	Downloads filtered results as CSV

3.3.2 Rider Detail View

Clicking any rider opens the full detail panel with tabbed sections:

TAB	DATA SHOWN	ACTIONS AVAILABLE
Profile	Name, phone, email, DOB, gender, address, profile photo	Edit profile fields, update preferred language
KYC Documents	Aadhaar (front/back), PAN, Driving License, bank documents — each with upload date, status badge	Approve / Reject individual documents with reason; bulk KYC decision
KYC History	Chronological log of all KYC actions — who did what, when, with notes	Read-only audit trail
Nominees	Nominee list with relationship, phone, Aadhaar (masked)	Add / edit nominees
Vehicle	Current vehicle allotment (if any) — vehicle reg number, hub, allotted date, daily rent	Link to vehicle detail
Wallet	Current balance, recent transactions	Link to full wallet page for this rider
Skills	Module completion progress, badges earned, gamification score	View detailed progress
Status Audit	Full history of rider status changes with timestamps and actor	Read-only compliance log

3.3.3 KYC Decision Workflow

> **IMPORTANT**
> KYC decisions are irreversible once set to VERIFIED. Rejection requires a written reason which is logged and visible to the rider.

The KYC Decision dropdown allows admins to set the overall KYC verdict:

DECISION	SETS RIDER.KYC_STATUS TO	NEXT ALLOWED ACTION
APPROVE	VERIFIED	Rider can proceed to training; eligible for vehicle allotment
REJECT	REJECTED	Rider must re-submit documents; reason is recorded in KYC log
REQUEST INFO	UNDER_REVIEW	Rider notified to provide clarification or re-upload specific documents

3.4 — Fleet Module

The Fleet module gives operations teams complete visibility and control over the electric vehicle fleet across all hubs.

3.4.1 Vehicles List View — Filters & Columns

FILTER / COLUMN	TYPE	OPTIONS	PURPOSE
`Registration Search`	Text input	Partial match on reg. number	Quick lookup by plate number
`Status Filter`	Dropdown	AVAILABLE, ALLOCATED, MAINTENANCE, RETIRED, LOST	Filter by operational state
`Hub Filter`	Dropdown	All active hubs	Show vehicles belonging to a hub
`City Filter`	Dropdown	All cities	City-scoped operations
`Compliance Alerts`	Checkbox toggle	ON/OFF	Show only vehicles with expired insurance/PUC/fitness
`Battery Below %`	Number input	0–100	Find vehicles with low battery needing charge

3.4.2 Vehicle Detail — Sections

SECTION	DATA FIELDS	ACTIONS
`Identity`	Registration, chassis, motor number, make, model, year, color	Edit vehicle details
`EV Specs`	Battery capacity (kWh), battery health (%), range (km), max speed	Update battery health %
`Live Telemetry`	Current GPS lat/lng (map link), battery level, odometer, charging status, last seen timestamp	View GPS history
`Compliance`	Insurance expiry, PUC expiry, fitness expiry — color-coded (red=expired, amber=30 days, green=OK)	Upload renewal documents
`Allotment History`	All rider allotments — rider name, dates, km driven, condition	Create new allotment
`Maintenance`	Upcoming service date/km, maintenance log history	Log maintenance record
`Status Change`	Current status with audit trail	Change status (AVAILABLE → MAINTENANCE, etc.)

3.4.3 Vehicle Allotment Form — Fields

FIELD	REQUIRED	DESCRIPTION
`Rider`	Yes	Search active riders — must have status=ACTIVE and no current allotment
`Vehicle`	Yes	Select AVAILABLE vehicle from hub's inventory
`Odometer at Allotment`	Yes	Current odometer reading — forms baseline for km_driven calculation
`Battery % at Allotment`	Yes	Battery percentage at time of handover
`Condition at Allotment`	Yes	Description of vehicle condition — photographic evidence recommended
`Daily Rent Amount`	Yes	INR amount per day — auto-filled from contract but editable
`Security Deposit`	Yes	One-time deposit amount collected from rider
`Expected Return Date`	No	Optional expected return date for planning

3.4.4 Hub Utilization Dashboard

Each hub shows a utilization card with:

- **Total Capacity** — configured vehicle slots at this hub
- **Available** — count of AVAILABLE vehicles (ready to allot)
- **Allocated** — count of ALLOCATED vehicles (currently with riders)
- **In Maintenance** — vehicles being serviced
- **Utilization %** — Allocated / (Allocated + Available) × 100
- **Manager** — Hub manager name and contact

3.5 — Payments Module

The Payments module provides complete financial oversight — wallet balances, rent scheduling, transaction ledger, overdue tracking, and manual adjustments.

3.5.1 Wallet View — Per Rider

SECTION	DATA SHOWN	AVAILABLE ACTIONS
`Wallet Summary`	Current balance, total earned (all time), total paid out, pending dues, security deposit held	—
`Ledger Entries`	All credit/debit transactions with timestamp, type, amount, balance_after	Filter by direction (Credit/Debit), payment_type, date range
`Rent Schedule`	Day-by-day rent dues with status (PENDING, PAID, OVERDUE, WAIVED)	Waive a rent entry with reason
`Admin Adjustment`	—	Credit or debit arbitrary amount with mandatory reason (audit logged)
`Incentive Credit`	—	Credit performance incentive with type and description

3.5.2 Ledger Filter Options

FILTER	OPTIONS
`Direction`	Credit (C), Debit (D), Both
`Payment Type`	DAILY_RENT, SECURITY_DEPOSIT, INCENTIVE, PENALTY, REFUND, ADJUSTMENT, WITHDRAWAL, TOPUP, DEPOSIT_REFUND
`Date Range`	From date — To date (accounting_date)
`Amount Range`	Min / Max INR amount

3.5.3 Payments Admin Summary

The admin summary dashboard (accessible to SUPER_ADMIN and CITY_ADMIN) shows platform-wide metrics:

- Total rent collected today / this week / this month
- Total overdue amount across all riders
- Count of riders with overdue rent (by days overdue: 1–3, 4–7, 7+ days)
- Incentives disbursed (MTD)
- Security deposits held vs refunded
- Top 10 riders by outstanding dues

3.6 — Marketplace Module

The Marketplace module connects client demand with available riders. It covers client management, demand slot lifecycle, rider applications, attendance, and payout.

3.6.1 Client Management

FIELD	DESCRIPTION
`Client Name`	Company name (e.g., "Zepto", "Blinkit", "Zomato")
`Category`	quick_commerce, food_delivery, logistics, etc.
`GSTIN / PAN`	Tax identifiers for invoice generation
`Primary Contact`	Name, email, phone of the client's operations contact
`Dark Stores`	Locations/warehouses where riders report for duty — each with address, GPS, city, hub linkage
`Contracts`	Pricing contracts defining pay structure, SLA terms, contract period per dark store

3.6.2 Demand Slot — Creation Form

FIELD	REQUIRED	OPTIONS / FORMAT	DESCRIPTION
`Client`	Yes	Dropdown — active clients	Which client is posting this demand
`Dark Store`	Yes	Dropdown — client's dark stores	Location where riders will report
`Shift Date`	Yes	Date picker	Date of the shift
`Shift Type`	Yes	MORNING (6am-2pm), AFTERNOON (2pm-10pm), NIGHT (10pm-6am), CUSTOM	Pre-defined time windows or custom
`Start / End Time`	Yes (if CUSTOM)	Time picker	Exact shift window
`Riders Required`	Yes	Integer (1–500)	How many riders the client needs
`Pay Structure`	Yes	PER_ORDER, PER_SHIFT, PER_HOUR, HYBRID	How rider earnings are calculated
`Pay Rate`	Yes	INR amount per applicable unit	e.g., ₹50/order or ₹500/shift
`Earnings Estimate`	No	INR	Shown to riders to set expectation
`Min Reliability Score`	No	0.0–5.0	Only show to riders with score above threshold
`Required Hubs`	No	Multi-select hubs	Target riders from specific hubs only
`Badge Required`	No	Badge code dropdown	Require a specific skill badge

3.6.3 Demand Slot Lifecycle — Status & Actions

STATUS	MEANING	AVAILABLE ADMIN ACTIONS
`DRAFT`	Created but not visible to riders	Edit, Publish, Cancel
`PUBLISHED`	Visible to riders in app — accepting applications	View applications, Bulk-confirm, Cancel
`PARTIALLY_FILLED`	Some riders confirmed but still need more	View applications, Bulk-confirm remaining
`FILLED`	All required riders confirmed	View final roster, Trigger payout post-shift
`EXPIRED`	Shift date passed without being filled	View analytics, Duplicate for new date
`CANCELLED`	Manually cancelled by admin	View audit log

3.6.4 Applications View — Columns & Actions

COLUMN	DESCRIPTION
`Rider Name / Phone`	Rider who applied
`Match Score`	AI-computed match score (0–100) based on hub proximity, reliability score, badges, past attendance
`Status`	APPLIED → SHORTLISTED → CONFIRMED → COMPLETED / NO_SHOW
`Applied At`	Timestamp of application
`Check-in / Check-out`	Actual attendance timestamps with GPS coordinates
`Orders Completed`	For PER_ORDER slots — number of deliveries done
`Hours Worked`	Actual hours from check-in to check-out
`Earnings`	Calculated earnings; credited flag
`Actions`	CONFIRM, REJECT, MARK NO-SHOW (post-shift)

3.7 — Skills Module

The Skills module allows admins to create and publish training content, track rider progress, and manage the gamification system.

3.7.1 Module Management

FIELD	DESCRIPTION
`Title (EN)`	Module title in English
`Title (HI)`	Module title in Hindi — shown to riders with language preference=Hindi
`Description`	Brief description of the module content
`Thumbnail URL`	Cover image for the module card in the app
`Sequence Order`	Display order in the module list (drag-to-reorder)
`Is Mandatory`	If ON, riders must complete this module before they can be activated
`Is Published`	Controls visibility in rider app — toggle to publish/unpublish

3.7.2 Video Management

FIELD	DESCRIPTION
`Video URL`	S3/MinIO path to the video file
`Duration (seconds)`	Video length — shown to riders as "Xm Ys"
`Points Reward`	Gamification points awarded on completion (default: 10)
`Has Quiz`	Toggle to enable a quiz after video completion
`Quiz Questions`	JSON array: [{question, options: [A,B,C,D], answer: "A"}]
`Quiz Pass Score`	Minimum correct answers % to earn points (default: 70%)

3.7.3 Leaderboard & Progress Tracking

The Skills analytics view shows:

- **Completion rates** per module — % of active riders who completed
- **Average quiz score** per module
- **Global leaderboard** — top 50 riders by total points
- **Badge distribution** — how many riders earned each badge
- **Onboarding readiness** — % of KYC-verified riders who completed mandatory training

3.8 — Support Module

The Support module is the agent workspace for managing rider support tickets from creation to resolution.

3.8.1 Ticket Queue — Filters

FILTER	OPTIONS	PURPOSE
`Status`	OPEN, ASSIGNED, IN_PROGRESS, WAITING_RIDER, RESOLVED, CLOSED, ESCALATED	Focus on actionable tickets
`Priority`	LOW, MEDIUM, HIGH, CRITICAL	Triage by urgency
`Category`	VEHICLE_ISSUE, PAYMENT_ISSUE, CUSTOMER_COMPLAINT, APP_ISSUE, KYC_QUERY, GENERAL, OTHER	Route to correct team
`Assigned To`	Agent name dropdown	View a specific agent's queue
`SLA Breached`	Yes / No toggle	Find tickets that missed SLA deadlines
`Date Range`	Created from/to	Time-bounded reporting

3.8.2 Ticket Detail — Sections

SECTION	CONTENT	ACTIONS
`Header`	Ticket number, status badge, priority, SLA countdown timer	Change priority, escalate
`Rider Info`	Rider name, phone, status — quick link to rider detail	—
`References`	Linked vehicle, allotment, demand slot, or transaction (if provided at creation)	Navigate to linked entity
`Message Thread`	Chronological in-app messages (RIDER, AGENT, SYSTEM) + WhatsApp messages	Reply, attach files, add internal note
`Assignment`	Current assignee and assignment history	Assign to self or another agent; bulk-assign from queue
`Resolution`	Resolution notes, resolved_at timestamp	Mark resolved with notes
`Rating`	Rider satisfaction score (1–5 stars) — shown after resolution	—

INTERNAL NOTES

Agents can post internal notes (is_internal=True) that are visible only to other agents, not to the rider. Use this for team handoff notes or sensitive operational details.

3.9 — Maintenance Module

The Maintenance module tracks all vehicle service events — preventive and corrective — along with parts costs, downtime, vendor management, and compliance alerts.

3.9.1 Maintenance Log — Create Form

FIELD	REQUIRED	OPTIONS
`Vehicle`	Yes	Search by registration number
`Maintenance Type`	Yes	PREVENTIVE, CORRECTIVE, BATTERY_SWAP, TYRE_CHANGE, BRAKE_SERVICE, ELECTRICAL, BODY_REPAIR, INSPECTION
`Scheduled Date`	Yes	Date picker
`Description`	Yes	Free text — what was done
`Parts Replaced`	No	Dynamic list: Part name, quantity, unit cost
`Labour Cost`	Yes	INR amount
`Odometer at Service`	Yes	Current km reading
`Next Service (km)`	No	Odometer reading for next scheduled service
`Next Service (date)`	No	Calendar date for next service
`Vendor`	No	Service vendor name
`Invoice URL`	No	Upload invoice document to MinIO
`Downtime Hours`	Yes	How long vehicle was out of service

3.9.2 Alert Severity Levels

SEVERITY	EXAMPLES	EXPECTED RESPONSE TIME
`CRITICAL`	Brake failure, battery fire risk, major electrical fault	Immediate — vehicle must be taken off road
`HIGH`	Battery health below 60%, insurance expired, failed fitness	Same day
`MEDIUM`	Overdue service, tyre wear, PUC expiring in 7 days	Within 3 days
`LOW`	Minor body damage, interior wear, recommended check	Within 2 weeks

SECTION 4

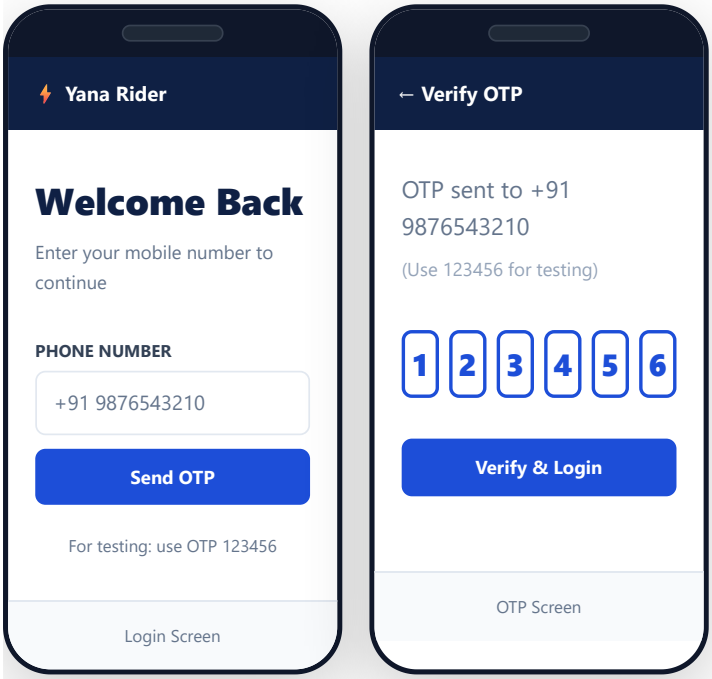
Rider Mobile App — User Manual

Complete guide to the Flutter mobile application for riders on Android and iOS.

QUICK START

Download the APK or install from the app store. For testing: use any 10-digit phone number and OTP `123456`. The app connects to the backend at `http://10.0.2.2:8000` (Android emulator) or a custom IP set via `API_BASE_URL`.

4.1 — Authentication Flow



Authentication Flow Steps

1 RIDER

Enter Phone Number

10-digit Indian mobile number. Tapping "Send OTP" calls `POST /api/v1/auth/rider/send-otp`. The auth service auto-creates a rider record if the phone is new.

2 SYSTEM

OTP Generated & Sent

Backend generates a 6-digit OTP. In development, OTP is logged to server console. In production, sent via SMS. Testing OTP: `123456`.

3 RIDER

Enter 6-digit OTP

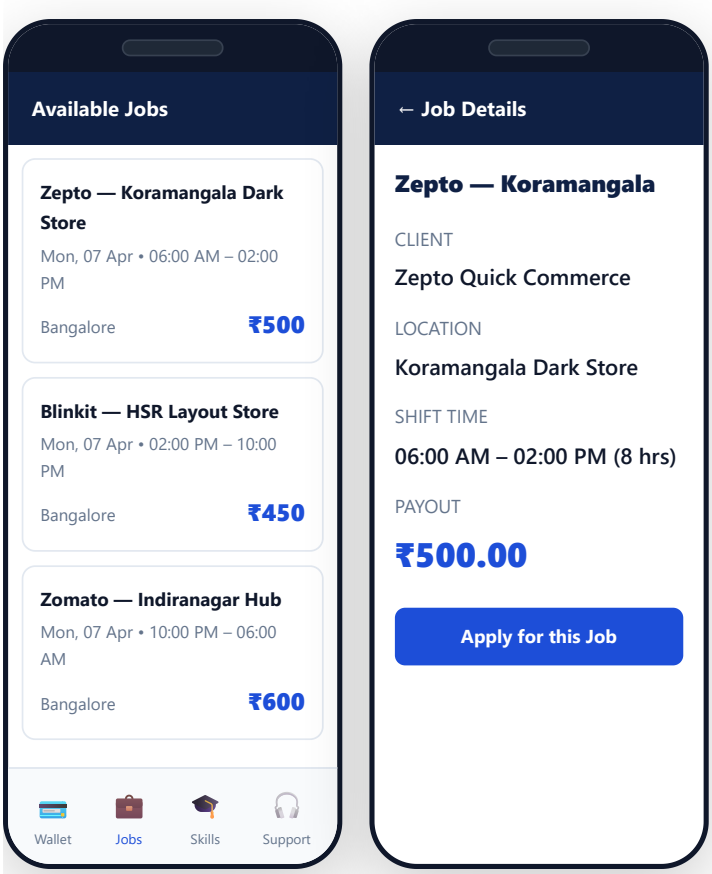
Uses Pinput widget — tapping the last digit automatically triggers verification. Calls `POST /api/v1/auth/rider/verify-otp`.

4 SYSTEM

Tokens Stored, Redirect to Home

Access token and refresh token stored in `FlutterSecureStorage` (encrypted on device). Rider ID stored separately. App redirects to `/home`.

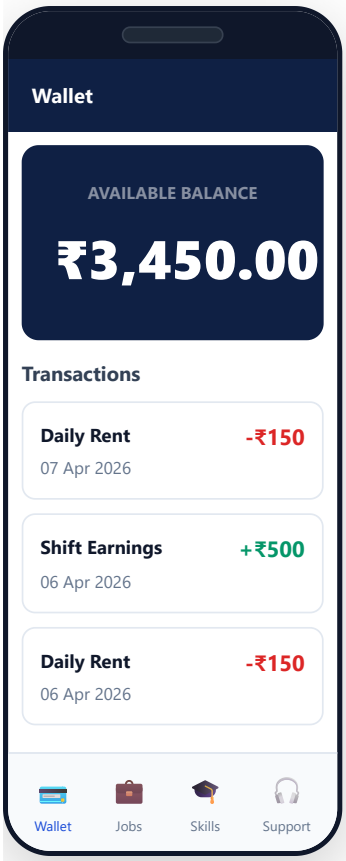
4.2 — Jobs / Demand Slots



Jobs Screen — Features

FEATURE	DESCRIPTION	API CALL
`Job List`	Shows all PUBLISHED demand slots visible to this rider (filtered by city, hub, score)	`GET /api/v1/marketplace/slots/?status=PUBLISHED`
`Pull-to-Refresh`	Swipe down to reload latest available jobs from server	Invalidates Riverpod provider
`Job Card`	Shows client name, dark store, date, shift time, payout amount	—
`Job Detail`	Tap card to view full details — location, pay structure, duration	—
`Apply Button`	Single-tap to submit application. Loading state during API call.	`POST /api/v1/marketplace/slots/{id}/apply/`

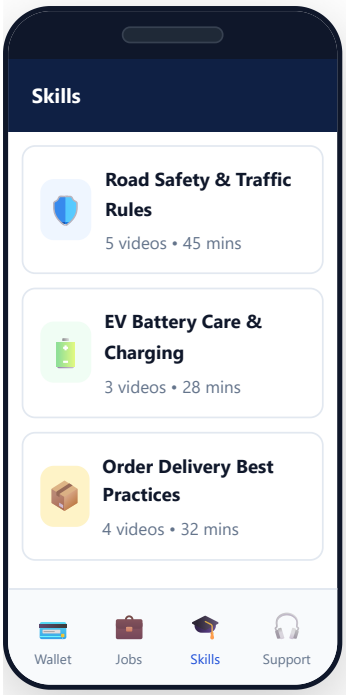
4.3 — Wallet



Wallet Features

FEATURE	DESCRIPTION	API CALL
`Balance Display`	Large balance card showing current wallet balance in INR	`GET /api/v1/payments/wallets/{rider_id}/`
`Transaction List`	All ledger entries — Credits shown in green (+), Debits in red (-) with payment type and date	`GET /api/v1/payments/wallets/{rider_id}/ledger/`
`Pull-to-Refresh`	Swipe down to reload both balance and transaction list	Invalidates both providers
`Credit Indicator`	Green downward arrow for credits (earnings, incentives, refunds)	direction == 'C'
`Debit Indicator`	Red upward arrow for debits (daily rent, penalties, withdrawals)	direction == 'D'

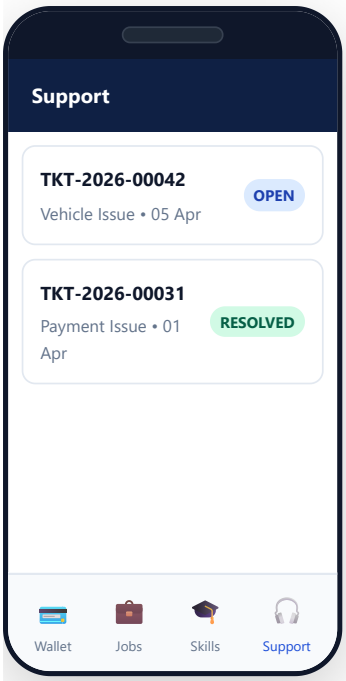
4.4 — Skills & Training



Skills Features

FEATURE	DESCRIPTION
`Module List`	All published training modules with thumbnail, title, video count and estimated duration
`Video Player`	Tap a video to expand the Chewie player inline. Auto-play starts. Progress tracked in real-time.
`Completion Tracking`	On video end, app calls `POST /api/v1/skills/videos/{id}/watch/` to credit points and mark complete
`Quiz`	If module has a quiz, it appears after last video. Multiple choice questions. Submit for instant scoring.
`Badge Earned`	On module completion, if badge criteria met, a congratulations dialog appears with badge name and icon
`Multilingual`	Titles and descriptions shown in rider's preferred language (EN/HI)

4.5 — Support



Create Ticket — Fields

FIELD	TYPE	OPTIONS
`Category`	Dropdown	Vehicle Issue, Payment Issue, Customer Complaint, App Issue, KYC Query, General, Other
`Subject`	Text field	Brief one-line description of the issue (max 500 chars)
`Description`	Multi-line text	Detailed explanation of the problem
`Submit`	Button	Calls `POST /api/v1/support/tickets/`. On success, returns to ticket list.

Bottom Navigation Bar

TAB	ICON	SCREEN	DATA LOADED
`Wallet`	account_balance_wallet	WalletScreen	Wallet balance + ledger entries
`Jobs`	work	JobsScreen	Published demand slots
`Skills`	school	SkillsScreen	Published training modules
`Support`	headset_mic	SupportScreen	Rider's support tickets

The app uses `IndexedStack` to preserve screen state when switching tabs — avoiding unnecessary API re-fetches.

SECTION 5

Data Architecture & Database Schemas

Complete schema documentation for every service, table, and field.

> **DATABASE**
> All services share a single PostgreSQL 15 instance. Each service manages its own tables through Django migrations. UUID primary keys are used throughout for distributed-safe identifiers. Soft deletes via `deleted_at` field preserve audit history.

5.1 — Core Tables Summary

TABLE	SERVICE	PRIMARY KEY	KEY RELATIONSHIPS	ROW VOLUME (EST.)
`admin_users`	Auth	UUID	—	Low (10–1000)
`riders`	Rider	UUID	hub, city, referred_by	Medium (1K–100K)
`rider_documents`	Rider	UUID	rider (FK)	Medium (5K–500K)
`rider_nominees`	Rider	UUID	rider (FK)	Medium
`kyc_verification_logs`	Rider	UUID	rider, document (FK)	High (append-only)
`cities`	Fleet	UUID	—	Very Low
`fleet_hubs`	Fleet	UUID	city (FK)	Low
`vehicles`	Fleet	UUID	hub (FK)	Medium
`vehicle_allotments`	Fleet	UUID	vehicle, hub (FK); rider_id	High
`vehicle_gps_telemetry`	Fleet	UUID	vehicle (FK)	Very High (partition by month)
`rider_wallets`	Payments	UUID	rider_id (unique)	= riders count
`wallet_ledger`	Payments	UUID	wallet (FK)	Very High (append-only)
`rent_schedule`	Payments	UUID	allotment_id; wallet_ledger (FK)	High
`payment_transactions`	Payments	UUID	ledger_entry (OneToOne)	High
`clients`	Marketplace	UUID	—	Low
`demand_slots`	Marketplace	UUID	client, dark_store (FK)	Medium (daily)
`demand_applications`	Marketplace	UUID	demand_slot (FK); rider_id	High
`skill_modules`	Skills	UUID	—	Very Low
`skill_videos`	Skills	UUID	module (FK)	Low
`rider_skill_progress`	Skills	UUID	rider_id; module, video (FK)	High
`rider_badges`	Skills	UUID	rider_id	Medium
`support_tickets`	Support	UUID	rider_id; assigned_to_id	High
`ticket_messages`	Support	UUID	ticket (FK)	Very High
`maintenance_logs`	Maintenance	UUID	vehicle (FK)	Medium

5.2 — Key Field Reference: Riders Table

FIELD	TYPE	NULLABLE	DESCRIPTION
`id`	UUID	No	Primary key — auto-generated
`phone`	varchar(15)	No	Unique — used as login identifier
`full_name`	varchar(200)	No	Legal name as per Aadhaar
`status`	varchar(30)	No	State machine: APPLIED → ACTIVE → OFFBOARDED
`kyc_status`	varchar(30)	No	PENDING → VERIFIED / REJECTED
`hub_id`	UUID	Yes	Assigned hub (set on activation)
`city_id`	UUID	Yes	Operating city
`aadhaar_number`	text (encrypted)	Yes	Encrypted at service layer before storage
`reliability_score`	decimal(3,2)	Yes	0.0–5.0 computed score used for marketplace matching
`deleted_at`	datetime	Yes	Soft delete — NULL means active record

5.3 — Key Field Reference: Wallet Ledger

FIELD	TYPE	DESCRIPTION
`payment_type`	varchar	DAILY_RENT, SECURITY_DEPOSIT, INCENTIVE, PENALTY, REFUND, ADJUSTMENT, WITHDRAWAL, TOPUP, DEPOSIT_REFUND
`amount`	decimal(12,2)	Always positive — direction determines credit or debit
`direction`	varchar(1)	'C' = Credit (money in), 'D' = Debit (money out)
`balance_after`	decimal(12,2)	Wallet balance snapshot after this transaction — enables ledger audit
`reference_id`	UUID	Links to business object (allotment_id, demand_application_id, etc.)
`accounting_date`	date	IST calendar date for financial reporting (independent of created_at UTC)
`gateway_txn_id`	text	Razorpay payment ID — indexed for webhook reconciliation

IMMUTABLE LEDGER POLICY

The `wallet_ledger` table is append-only. No row is ever updated or deleted. Corrections are made by posting a reversal entry with payment_type=REFUND or ADJUSTMENT. This ensures a tamper-proof financial audit trail.

SECTION 6

Data Relationships

How entities connect across services and what those connections enable.

6.1 — Cross-Service Entity Map

Because Yana OS uses microservices, cross-service references are stored as raw **UUID fields** (not Django ForeignKeys), allowing services to remain independently deployable. The gateway composes data when a full view is needed.

ENTITY A	ENTITY B	RELATIONSHIP	HOW LINKED	BUSINESS MEANING
`Rider`	FleetHub	Many-to-One	riders.hub_id → fleet_hubs.id	A rider belongs to one hub; a hub manages many riders
`Vehicle`	FleetHub	Many-to-One	vehicles.hub_id → fleet_hubs.id	Each vehicle is assigned to a hub for management
`VehicleAllotment`	Rider	Many-to-One	allotments.rider_id (UUID field)	A rider can have at most one ACTIVE allotment at a time
`VehicleAllotment`	Vehicle	Many-to-One	allotments.vehicle_id → vehicles.id	A vehicle has one active allotment at a time (enforced by unique constraint)
`RiderWallet`	Rider	One-to-One	rider_wallets.rider_id (unique UUID)	Every active rider has exactly one wallet
`WalletLedger`	RentSchedule	One-to-One	rent_schedule.paid_ledger_id	When rent is collected, ledger entry is linked to the schedule entry
`RentSchedule`	VehicleAllotment	Many-to-One	rent_schedule.allotment_id (UUID)	Daily rent entries generated per active allotment
`DemandApplication`	DemandSlot	Many-to-One	demand_applications.demand_slot_id	Many riders apply for one slot
`DemandApplication`	Rider	Many-to-One	demand_applications.rider_id (UUID)	One rider can apply to many slots (not same slot twice — unique constraint)
`DemandSlot`	Client	Many-to-One	demand_slots.client_id	A client posts many demand slots
`SupportTicket`	Rider	Many-to-One	support_tickets.rider_id (UUID)	A rider can have many tickets over time
`SupportTicket`	Vehicle/Allotment	Optional Many-to-One	support_tickets.vehicle_id / allotment_id	Ticket can be linked to specific vehicle or allotment for context
`RiderSkillProgress`	Rider	Many-to-One	rider_skill_progress.rider_id (UUID)	Tracks each rider's per-video progress
`RiderBadge`	Rider	Many-to-One	rider_badges.rider_id (UUID)	A rider can earn multiple badges; each badge earned once
`MaintenanceLog`	Vehicle	Many-to-One	maintenance_logs.vehicle_id	Full service history per vehicle

6.2 — Rider 360° Data View

The admin dashboard's rider detail view composes data from multiple services to show a complete rider profile:

DATA SHOWN	SOURCE SERVICE	API CALL
Profile & KYC status	Rider Service	`GET /api/v1/riders/{id}/`
KYC documents & logs	Rider Service	`GET /api/v1/riders/{id}/kyc/logs/`
Active vehicle allotment	Fleet Service	`GET /api/v1/fleet/allotments/?rider_id={id}`
Wallet balance & ledger	Payments Service	`GET /api/v1/payments/wallets/{id}/`
Rent schedule & overdue	Payments Service	`GET /api/v1/payments/rent/{id}/schedule/`
Job applications history	Marketplace Service	`GET /api/v1/marketplace/riders/{id}/applications/`
Training progress & badges	Skills Service	`GET /api/v1/skills/riders/{id}/progress/`
Support tickets	Support Service	`GET /api/v1/support/riders/{id}/tickets/`

6.3 — Data Flow: Allotment → Rent → Payment

This shows the financial data chain triggered by a vehicle allotment:

FLOW

```
1. HUB_OPS creates allotment → VehicleAllotment created (status=ACTIVE)
   VehicleAllotment { rider_id, vehicle_id, daily_rent_amount=150, security_deposit=2000 }

2. Celery task fires → generates RentSchedule entries for 30 days
   RentSchedule × 30 rows { allotment_id, due_date, amount=150, status=PENDING }

3. Also fires → WalletLedger entry for security deposit
   WalletLedger { payment_type=SECURITY_DEPOSIT, amount=2000, direction=D }

4. Nightly Celery beat → processes due rent entries
   For each RentSchedule WHERE due_date=today AND status=PENDING:
     → Attempt UPI autopay via Razorpay API
     → On success:
         WalletLedger { payment_type=DAILY_RENT, amount=150, direction=D, balance_after=X }
         PaymentTransaction { status=SUCCESS, gateway_txn_id=pay_xxx }
         RentSchedule.status = PAID
     → On failure:
         RentSchedule.status = OVERDUE, days_overdue++, penalty applied

5. Vehicle returned → WalletLedger deposit refund entry
   WalletLedger { payment_type=DEPOSIT_REFUND, amount=2000, direction=C }
```

SECTION 6.5

Data Flow Diagrams

How data moves through the system for key operations.

Authentication Token Flow

SEQUENCE

```
Rider App          Nginx Gateway       Auth Service         PostgreSQL
    |                   |                   |                   |
```

```
    |── POST /send-otp ─►                 |                   |
    |                   ├── forward ──────►                   |
    |                   |                  ├── CREATE rider ──►
    |                   |                  ├── STORE otp hash ─►
    |                   |                  ◄── ok ────────────|
    |                   ◄── { OTP sent } ──|                   |
    ◄── { message } ────|                 |                   |
    |                   |                  |                   |
    |── POST /verify ──►                  |                   |
    |                   ├── forward ──────►                   |
    |                   |                  ├── VERIFY otp ─────►
    |                   |                  ├── GENERATE JWT ───|
    |                   ◄── { tokens } ────|                   |
    ◄── { access+refresh }                |                   |
    | store in SecureStorage              |                   |
    |                   |                  |                   |
    |── GET /wallet ───►                  |                   |
    |  (Bearer: access)  |                  |                   |
    |                   ├── validate JWT ──►                   |
    |                   ├── forward to payments-service         |
    |                   ◄── wallet data ──────────────────────|
    ◄── { balance }─────|
```

Demand Slot Application Flow

SEQUENCE

```
Rider App         Marketplace Service     Payments Service    Celery Worker
    |                   |                     |                 |
    |── GET /slots ─────►                     |                 |
    |                   ├── filter by status=PUBLISHED          |
    |                   ├── filter by rider city/hub            |
    |                   ├── filter by min_reliability_score     |
    ◄── [slot list] ────|                     |                 |
    |                   |                     |                 |
    |── POST /apply ────►                     |                 |
    |                   ├── CREATE DemandApplication            |
    |                   ├── run matching_score algorithm        |
    |                   ├── notify admin queue                  |
    ◄── { application } ─|                    |                 |
    |                   |                     |                 |
    |    [ADMIN confirms rider]               |                 |
    |                   ├── application.status = CONFIRMED      |
    |                   ├── slot.riders_confirmed++             |
    |                   |                     |                 |
    |    [Shift day - Rider checks in]        |                 |
    |── POST /check-in ─►                     |                 |
    |                   ├── record GPS, timestamp              |
    ◄── { ok } ─────────|                     |                 |
    |                   |                     |                 |
    |    [Post-shift - Rider checks out]      |                 |
    |── POST /check-out ─►                    |                 |
    |                   ├── application.status = COMPLETED      |
    ◄── { ok } ─────────|                     |                 |
    |                   |                     |                 |
    |        [Celery nightly task]                              |
    |                   ├──────────────────────────────── calculate_earnings
    |                   |                     ◄── WalletLedger CREDIT
    |                   |                     |   payment_type=INCENTIVE
    |                   ├── application.earnings_credited = True |
```

SECTION 7

Business Workflows

End-to-end operational playbooks for every key business process.

7.1 — Rider Onboarding Workflow

The onboarding workflow takes a new applicant from phone registration to an ACTIVE rider eligible for vehicle allotment and demand work. It spans 4 services.

1 RIDER

Phone Registration & OTP Login

Rider enters phone number in the app. Auth service sends OTP. Rider verifies → JWT tokens issued. Rider status: **APPLIED**

2 RIDER

Profile Completion

Rider fills name, email, DOB, gender, address, bank details via the app. Admin can also complete profile. Status: **APPLIED**

3 RIDER

Document Upload

Upload Aadhaar (front + back), PAN, Driving License (front + back), bank passbook/cancelled cheque, profile photo. Documents stored in MinIO. Status: **DOCS_SUBMITTED** → **KYC_PENDING**

4 ADMIN

Document Review & KYC Decision

SUPPORT_AGENT reviews each document. Can approve/reject individual documents with reason. Final KYC decision: VERIFIED or REJECTED. A rejected KYC logs the reason and notifies the rider. Status: **UNDER_REVIEW** → **KYC_VERIFIED** or **KYC_FAILED**

5 RIDER

Nominee Registration

Rider adds at least one nominee (family member) with relationship, phone, and Aadhaar. Required for insurance and emergency contact purposes.

6 RIDER

Mandatory Training

Rider completes all mandatory skill modules in the app (road safety, EV care, delivery best practices). Must pass quizzes. Skills service marks onboarding_readiness=True. Status: **TRAINING**

7 ADMIN

Rider Activation

HUB_OPS or CITY_ADMIN verifies the rider is physically at the hub. Calls `POST /api/v1/riders/{id}/activate/`. Rider status: **ACTIVE**. Rider is now eligible for vehicle allotment and demand slots.

8 SYSTEM

Wallet Created

On activation, Payments service auto-creates a `RiderWallet` with zero balance. Rider can now receive earnings and is subject to daily rent collection.

7.2 — Vehicle Allotment & Return Workflow

1 HUB_OPS

Select Vehicle & Rider

HUB_OPS navigates to Fleet → Allotments → New Allotment. Selects an AVAILABLE vehicle and an ACTIVE rider without a current allotment. Enters odometer, battery %, condition notes, daily rent, and security deposit amount.

2 SYSTEM

Allotment Created — Status Changes

VehicleAllotment created (status=ACTIVE). Vehicle status changed to ALLOCATED. Unique constraints enforced — cannot allot a vehicle already allocated or allot two vehicles to one rider simultaneously.

3 CELERY

Financial Setup

Celery task generates 30-day RentSchedule entries. Security deposit debited from rider's wallet (or invoiced if wallet insufficient). Nightly collection begins from next calendar day.

4 RIDER

Rider Operates Vehicle

Rider uses the EV for daily deliveries. GPS telemetry is ingested by the fleet-telemetry FastAPI sidecar (real-time). Battery level, location, and odometer updated continuously.

5 HUB_OPS

Vehicle Return

Calls `POST /api/v1/fleet/allotments/{id}/return/`. Records: odometer at return, battery %, condition at return, damage notes. Calculates km_driven = odometer_return − odometer_allotment. Allotment status: RETURNED. Vehicle status: AVAILABLE.

6 SYSTEM

Deposit Refund & Rent Schedule Closure

If no damage noted: security deposit refunded to wallet (WalletLedger: DEPOSIT_REFUND, Credit). If damage: deduct damage cost from deposit, refund remainder. Future RentSchedule entries marked WAIVED. Collection stops.

7.3 — Demand Slot & Earnings Workflow

1. SALES

Create Demand Slot

Sales team receives demand from client (e.g., Zepto needs 20 riders for tomorrow 6am–2pm at Koramangala). Creates slot: client, dark store, shift date/time, riders_required, pay_structure.

2. CITY_ADMIN

Publish Slot

Admin reviews the slot and publishes it. Status changes DRAFT → PUBLISHED. Slot is now visible in the rider app to eligible riders (filtered by city, hub, reliability score, badges).

3. RIDER

Apply for Slot

Rider views available jobs in the app, taps the Zepto slot card, reviews details (pay, timing, location), and taps "Apply". DemandApplication created with status=APPLIED. Matching engine calculates match_score.

4. CITY_ADMIN

Bulk Confirm Riders

Admin views applications sorted by match_score. Selects top applicants and bulk-confirms them. Application status: CONFIRMED. Slot riders_confirmed incremented. When riders_confirmed = riders_required, slot status: FILLED.

5. RIDER

Check-In at Dark Store

Day of shift: Rider arrives at Zepto dark store, taps Check-In in app. App records timestamp and GPS coordinates. These are validated against the dark store location to prevent proxy check-ins.

6. RIDER

Complete Shift & Check-Out

Rider completes deliveries (orders_completed tracked). At shift end, taps Check-Out. Application status: COMPLETED. hours_worked calculated. orders_completed finalized.

7. CELERY

Earnings Calculation & Payout

Post-shift Celery task calculates earnings: PER_ORDER slots: 15 orders × ₹50 = ₹750. PER_SHIFT: flat ₹500. Credits WalletLedger (payment_type=INCENTIVE, direction=C). Updates earnings_credited=True in application.

7.4 — Support Ticket Workflow

1 RIDER

Create Ticket

Rider selects category (Vehicle Issue, Payment Issue, etc.), enters subject and description. App calls `POST /api/v1/support/tickets/`. Ticket created with status=OPEN, priority=MEDIUM. Auto-assigned ticket number (TKT-YYYY-NNNNN).

2 SUPPORT_AGENT

Ticket Assignment

Agent sees new ticket in queue, assigns to themselves. Status: ASSIGNED. SLA timer starts. High/Critical priority tickets have shorter SLA windows (2h/4h vs 24h/48h for Low/Medium).

3 SUPPORT_AGENT

Investigation & Communication

Agent reviews ticket with linked entities (vehicle, allotment, payment). Posts messages visible to rider. Can add internal notes (is_internal=True) for team context. Status: IN_PROGRESS.

4 SUPPORT_AGENT

Resolution

Agent resolves the issue (issues refund, arranges vehicle swap, clarifies KYC). Posts resolution notes. Calls `POST /api/v1/support/tickets/{id}/resolve/`. Status: RESOLVED.

5 RIDER

Rating & Closure

Rider receives resolution notification, reviews, and rates satisfaction (1–5 stars). Ticket status: CLOSED. If unsatisfied, rider can escalate → Status: ESCALATED → Manager queue.

7.5 — Daily Rent Collection Workflow

1. CELERY BEAT (Nightly 00:01 IST)

   Daily Rent Collection Job Fires

   Celery beat scheduler triggers the `collect_daily_rents` task. Queries all RentSchedule entries where due_date=today AND status=PENDING.

2. SYSTEM

   Check Wallet Balance & UPI Mandate

   For each due entry: check if rider has active UPI mandate (razorpay_mandate_id). If mandate exists and balance sufficient → trigger Razorpay debit. If no mandate → create manual payment request.

3. RAZORPAY WEBHOOK

   Payment Gateway Callback

   Razorpay webhook fires on payment success/failure at `POST /api/v1/payments/webhooks/razorpay/`. Signature verified. On success: WalletLedger entry created, RentSchedule.status=PAID. On failure: status=OVERDUE.

4. SYSTEM

   Overdue Handling & Penalties

   Overdue entries accumulate `days_overdue`. After 3 days: penalty applied (overdue_penalty added). After 7 days: rider flagged for suspension review. After 14 days: force-return process initiated. All actions audit-logged.

SECTION 8

Technology Stack

All frameworks, libraries, and design patterns used across the platform.

8.1 — Backend Services

TECHNOLOGY	VERSION	USAGE	RATIONALE
Python	3.11+	All backend services	Rich ecosystem, type hints, async support
Django	4.2 LTS	7 of 8 services	Battle-tested ORM, admin, migrations — LTS ensures 2-year security support
Django REST Framework	3.14	All Django services	API views, serializers, authentication, pagination
FastAPI	0.109	fleet-telemetry only	High-throughput async I/O for GPS ingestion; WebSocket support
Celery	5.3	All services	Async task queue for OTP, rent collection, earnings payout, compliance checks
Redis	7	Celery broker + cache	Low-latency message queue; token caching; rate limiting counters
PostgreSQL	15	All services	ACID compliance, UUID support, JSON fields, partition support (for GPS telemetry)
MinIO	Latest	File storage	S3-compatible, self-hosted, on-prem data sovereignty for KYC documents
drf-spectacular	0.27	All services	Auto-generates OpenAPI 3.0 docs — accessible at `/api/docs/` per service

Backend Architectural Patterns

PATTERN	DESCRIPTION	APPLIED IN
Soft Deletes	`deleted_at` field on all core entities. No hard deletes in production. Queries filter WHERE deleted_at IS NULL.	All services
Audit Trail	Immutable log tables for status/decision changes: `KYCVerificationLog`, `VehicleStatusAudit`, `DemandSlotAudit`	Rider, Fleet, Marketplace
Immutable Ledger	Append-only financial ledger. No UPDATE/DELETE on ledger rows. Corrections via reversal entries.	Payments
Optimistic Locking	`version` field on RiderWallet prevents concurrent balance corruption in multi-worker Celery environment	Payments
Encryption at Rest	Sensitive PII (Aadhaar, PAN, bank account) encrypted at service layer using symmetric key before writing to DB	Rider, Fleet
UUID Primary Keys	All tables use UUID v4 PKs for distributed-safe generation without sequence locks	All services
Standardized Response	All endpoints return `{success, data}` or `{success:false, error:{message, code}}`	All services
Role-Based Access Control	JWT payload includes `role` and `city_id` / `hub_id`. Each view checks permissions before proceeding.	Auth + all services

8.2 — Admin Dashboard (React)

TECHNOLOGY	VERSION	USAGE
`React`	18.2	UI framework — hooks-based functional components throughout
`Vite`	5.1	Build tool — HMR for dev, optimized bundle for production
`Tailwind CSS`	3.4	Utility-first styling — no custom CSS files needed
`React Router DOM`	6.22	Client-side routing with protected route guards
`Axios`	1.6	HTTP client with interceptors for auth token injection & refresh
`Recharts`	2.12	Revenue charts, fleet status donuts, hub utilization bars
`TanStack Table`	8.13	Paginated, sortable, filterable data tables
`Lucide React`	0.344	Icon library — consistent iconography throughout
`React Hot Toast`	2.4	Non-blocking success/error notification toasts

Frontend Architecture

- **State:** React Context API + `useState` / `useEffect` hooks. No Redux needed at current scale.
- **Auth State:** JWT tokens in `localStorage`. Axios interceptor attaches Bearer token to every request. On 401, auto-refresh attempted. On refresh failure, redirect to login.
- **Routing:** Protected routes redirect unauthenticated users to `/login`. Role-based menu items hidden via role check in context.
- **Data Fetching:** Axios calls in `useEffect` with cleanup for stale request cancellation.
- **Forms:** Controlled components with local state. Validation before submission.

8.3 — Rider Mobile App (Flutter)

TECHNOLOGY	VERSION	USAGE
`Flutter`	3.41.5	Cross-platform mobile framework (Android + iOS from single codebase)
`Dart`	3.11.3	Programming language — strong typed, null-safe
`flutter_riverpod`	2.5.1	State management — providers for auth, wallet, jobs, skills, support
`riverpod_annotation`	2.3.5	Code generation annotations (@riverpod, @Riverpod)
`build_runner`	2.4.9	Generates `.g.dart` files for providers and models
`freezed`	2.5.2	Immutable model classes with copyWith, equality, serialization
`json_serializable`	6.8.0	Generates `fromJson` / `toJson` methods
`dio`	5.4.3	HTTP client with interceptors for JWT injection and silent token refresh
`go_router`	14.1.4	Declarative routing with auth redirect guard at `/login`
`flutter_secure_storage`	9.0.0	Encrypted keychain storage for access/refresh tokens
`video_player + chewie`	2.8.6 / 1.8.1	In-app video playback for training content
`pinput`	3.0.1	OTP input field with auto-submit on 6th digit
`cached_network_image`	3.3.1	Profile photos and module thumbnails with LRU disk cache
`intl`	0.19.0	Date/currency formatting — INR with ₹ symbol, locale-aware dates

App Architecture (MVVM with Riverpod)

LAYER	FILES	RESPONSIBILITY
`View (UI)`	`lib/features/*/screens/*.dart`	Flutter widgets. ConsumerWidget/ConsumerStatefulWidget. Only UI logic.
`Provider (ViewModel)`	`lib/features/*/providers/*.dart`	Riverpod providers. Orchestrates repository calls. Holds async state.
`Repository (Data)`	`lib/features/*/data/*.dart`	Dio HTTP calls. Maps JSON to models. One class per feature.
`Models`	`lib/features/*/models/*.dart`	Plain Dart classes with fromJson factories. Optional freezed for complex models.
`Core`	`lib/core/`	Dio client (auth interceptor), TokenStorage, AppConstants, AppRouter

SECTION 9

Infrastructure & Deployment

Docker, Kubernetes, and operational setup guide.

9.1 — Local Development Setup

Prerequisites

- Docker Desktop 24+ with Docker Compose v2
- Flutter SDK 3.41+ (for mobile app development)
- Node.js 20+ (for admin dashboard development)
- Android Studio / Xcode (for mobile emulators)

Start the Backend Stack

```bash
# Clone repository
git clone https://github.com/anirudhasi/yana-osV1.0.git
cd yana-osV1.0

# Start core services (auth, rider, fleet, payments, marketplace, nginx)
docker compose up --build

# Start optional services (skills, support, maintenance)
docker compose --profile optional up --build

# Start with admin dashboard
docker compose --profile optional --profile ui up --build

# View logs for a specific service
docker compose logs -f auth-service

# Run database migrations (auto-run on startup)
docker compose exec rider-service python manage.py migrate

# Create superuser (if needed)
docker compose exec auth-service python manage.py createsuperuser
```

Service URLs (After docker compose up)

SERVICE	URL	CREDENTIALS
Admin Dashboard	http://localhost:3000	admin@yana.in / Admin@123
API Gateway	http://localhost:8000	—
Swagger Docs (Auth)	http://localhost:8001/api/docs/	—
Swagger Docs (Rider)	http://localhost:8002/api/docs/	—
MinIO Console	http://localhost:9001	yana_admin / yana_secret_2024
PostgreSQL	localhost:5432	yana / yana_pass (internal)
Redis	localhost:6379	No auth (internal)

Start the Mobile App

```bash
# From the rider-app directory
cd rider-app

# Install dependencies
flutter pub get
```

```
# Generate provider/model code
dart run build_runner build --delete-conflicting-outputs

# Run on Android emulator (uses 10.0.2.2 to reach host)
flutter run

# Run on physical device (replace with your machine's LAN IP)
flutter run --dart-define=API_BASE_URL=http://192.168.1.100:8000

# Run on specific device
flutter devices
flutter run -d emulator-5554

# Build release APK
flutter build apk --release
```

Start the Admin Dashboard (Development)

```bash
cd admin-dashboard
npm install
npm run dev   # Starts at http://localhost:3000 with HMR
npm run build # Production build to dist/
```

Docker Compose Service Profiles

PROFILE	SERVICES INCLUDED	USE WHEN
(default / no profile)	postgres, redis, minio, auth, rider, fleet, payments, marketplace, nginx + their celery workers	Core platform development and testing
optional	maintenance, skills, support + their celery workers	Full feature set including training and support
ui	admin-dashboard	Running the React dashboard in a container

9.2 — Kubernetes Deployment

Production deployment uses Kubernetes manifests in the `/k8s/` directory and a Helm chart in `/helm/`.

Kubernetes Resources

RESOURCE TYPE	COUNT	DESCRIPTION
`Deployments`	16+	One per service + one per Celery worker + beat schedulers
`Services`	10+	ClusterIP for internal service discovery; LoadBalancer for nginx
`ConfigMaps`	8+	Environment variables per service (DB URL, Redis URL, etc.)
`Secrets`	5+	DB password, Redis auth, MinIO credentials, JWT secret keys, Razorpay keys
`Ingress`	1	Nginx Ingress Controller with rate limiting, TLS termination
`PersistentVolumeClaims`	3	postgres-data, redis-data, minio-data
`HorizontalPodAutoscaler`	8	Auto-scale services based on CPU (target 70%)
`ServiceAccounts / RBAC`	Configured	Least-privilege access within cluster

Deploy with Helm

BASH

```bash
# Install / upgrade the Yana OS Helm chart
helm upgrade --install yana-os ./helm \
  --set global.dbPassword=your_secure_password \
  --set global.jwtSecret=your_jwt_secret \
  --set payments.razorpayKey=rzp_live_xxx \
  --namespace yana --create-namespace

# Check rollout status
kubectl rollout status deployment/auth-service -n yana

# View all pods
kubectl get pods -n yana
```

CI/CD Pipeline

STAGE	ACTION	TOOL
`Lint`	Python flake8, Dart flutter analyze	GitHub Actions
`Test`	pytest (backend), flutter test (app)	GitHub Actions
`Build`	Docker image build per service	GitHub Actions
`Push`	Push images to container registry (GHCR/ECR)	GitHub Actions
`Deploy`	helm upgrade on tag push; canary on PR	Argo CD / GitHub Actions

---

SECTION 10

API Gateway & Routing

How Nginx routes requests, enforces rate limits, and handles special protocols.

10.1 — Route Table

URL PREFIX	UPSTREAM SERVICE	PORT	NOTES
`/api/v1/auth/`	auth-service	8001	JWT login, OTP, token refresh
`/api/v1/riders/`	rider-service	8002	Rider CRUD, KYC, documents, nominees
`/api/v1/fleet/`	fleet-service	8003	Vehicles, hubs, allotments, GPS history
`/api/v1/payments/`	payments-service	8004	Wallets, ledger, rent, transactions
`/api/v1/marketplace/`	marketplace-service	8005	Clients, demand slots, applications
`/api/v1/maintenance/`	maintenance-service	8006	Maintenance logs and alerts
`/api/v1/skills/`	skills-service	8007	Modules, videos, progress, badges
`/api/v1/support/`	support-service	8008	Tickets, messages, analytics
`/telemetry/`	fleet-telemetry (FastAPI)	8013	High-throughput GPS ingestion
`/ws/`	fleet-telemetry (WebSocket)	8013	Real-time GPS WebSocket feed
`/demo/`	Static files	—	Browser demo UI
`/health`	Nginx internal	—	Gateway health check for load balancers

10.2 — Rate Limiting

ENDPOINT	LIMIT	SCOPE	REASON
`/api/v1/auth/rider/send-otp`	5 req/min	Per IP	OTP spam prevention — prevents mass phone number enumeration
`All other` `/api/v1/`	100 req/min	Per IP	General API abuse prevention
`/telemetry/`	500 req/min	Per IP	High-frequency GPS push from IoT devices

10.3 — Request Headers

HEADER	VALUE	PURPOSE
`Authorization`	Bearer <JWT token>	Every authenticated request must include this
`Content-Type`	application/json	All POST/PATCH/PUT requests
`X-Real-IP`	Set by Nginx	Client's real IP forwarded to services for audit logging
`X-Forwarded-For`	Set by Nginx	Full proxy chain for services behind multiple proxies

10.4 — Nginx Configuration Highlights

NGINX

```
# Max upload size (for KYC document uploads)
client_max_body_size 20M;
```

```
# Timeouts
proxy_connect_timeout 30s;
proxy_read_timeout    60s;   # 60s for long-running report queries

# WebSocket upgrade for fleet telemetry
location /ws/ {
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_pass http://fleet-telemetry:8013;
}
```

APPENDIX

Reference: Status Enumerations

All system-wide status values and their meanings.

Rider Status State Machine

All Enumerations Reference

ENUM GROUP	VALUES
`Rider Status`	APPLIED → DOCS_SUBMITTED → KYC_PENDING → KYC_FAILED → VERIFIED → TRAINING → ACTIVE → SUSPENDED → OFFBOARDED
`KYC Status`	PENDING → SUBMITTED → UNDER_REVIEW → VERIFIED / REJECTED
`Admin Roles`	SUPER_ADMIN, CITY_ADMIN, HUB_OPS, SALES, SUPPORT_AGENT, VIEWER
`Vehicle Status`	AVAILABLE, ALLOCATED, MAINTENANCE, RETIRED, LOST
`Allotment Status`	ACTIVE, RETURNED, FORCE_RETURNED, EXPIRED
`Demand Slot Status`	DRAFT, PUBLISHED, PARTIALLY_FILLED, FILLED, EXPIRED, CANCELLED
`Application Status`	APPLIED, SHORTLISTED, CONFIRMED, REJECTED, WITHDRAWN, NO_SHOW, COMPLETED
`Shift Type`	MORNING (6am-2pm), AFTERNOON (2pm-10pm), NIGHT (10pm-6am), CUSTOM
`Pay Structure`	PER_ORDER, PER_SHIFT, PER_HOUR, HYBRID
`Payment Type`	DAILY_RENT, SECURITY_DEPOSIT, INCENTIVE, PENALTY, REFUND, ADJUSTMENT, WITHDRAWAL, TOPUP, DEPOSIT_REFUND
`Transaction Status`	PENDING, PROCESSING, SUCCESS, FAILED, REVERSED
`Ledger Direction`	C (Credit — money in), D (Debit — money out)
`Rent Schedule Status`	PENDING, PAID, OVERDUE, WAIVED, PARTIAL
`Ticket Status`	OPEN, ASSIGNED, IN_PROGRESS, WAITING_RIDER, RESOLVED, CLOSED, ESCALATED
`Ticket Priority`	LOW, MEDIUM, HIGH, CRITICAL
`Ticket Category`	VEHICLE_ISSUE, PAYMENT_ISSUE, CUSTOMER_COMPLAINT, APP_ISSUE, KYC_QUERY, GENERAL, OTHER
`Sender Type`	RIDER, AGENT, SYSTEM
`Badge Codes`	SAFETY_FIRST, SPEED_DEMON, CONSISTENT, VETERAN, TOP_EARNER, QUICK_LEARNER, COMPLIANT
`Alert Severity`	LOW, MEDIUM, HIGH, CRITICAL
`Maintenance Type`	PREVENTIVE, CORRECTIVE, BATTERY_SWAP, TYRE_CHANGE, BRAKE_SERVICE, ELECTRICAL, BODY_REPAIR, INSPECTION
`Maintenance Status`	SCHEDULED, IN_PROGRESS, COMPLETED, CANCELLED
`Document Type`	AADHAAR_FRONT, AADHAAR_BACK, PAN, DRIVING_LICENSE_FRONT, DRIVING_LICENSE_BACK, BANK_PASSBOOK, CANCELLED_CHEQUE, PROFILE_PHOTO, OTHER
`Payment Gateway`	razorpay, cashfree, upi_autopay, manual

Seeded Test Accounts

EMAIL	PASSWORD	ROLE	USE FOR
admin@yana.in	Admin@123	SUPER_ADMIN	Full platform access — all modules
ops@yana.in	Ops@123	HUB_OPS	Fleet operations — allotments, returns, maintenance
sales@yana.in	Sales@123	SALES	Client and demand slot management

Rider app testing: Any 10-digit phone number with OTP `123456`